

# Лекция 1.1. Введение

**Хронометраж пары (90 мин):** вход в курс ~5 мин → информация и измерение ~35 мин → системы счисления ~35 мин → мост к курсу ~10 мин → закрепление ~5 мин.

## Цели занятия

После лекции вы умеете:

1. Отличать информацию (снятие неопределённости) от данных (зафиксированная последовательность символов или битов).
2. Считать количество информации по содержательному подходу для равновозможных исходов ( $I = \log_2 N$ ) и по алфавитному ( $I = n \log_2 k$ ).
3. Объяснить, зачем компьютеру бит и байт, и переводить единицы объёма с оговоркой 1000 vs 1024.
4. Записать целое число в системах с основанием 2, 8, 10, 16 и перевести его в другую систему **через десятичную**.
5. Связать «1 байт = 8 бит» с числом различных кодов и понять, зачем человеку шестнадцатеричная запись.

---

## 0. Зачем это в курсе

Компьютер не «понимает смысл». Он хранит и преобразует **коды**: последовательности нулей и единиц, из которых собраны числа, текст, изображения, таблицы признаков.

Весь семестр мы будем ходить по трём этажам:

1. **Информация и данные** — что кодируем.
2. **Алгоритм** — что делаем с данными.
3. **Программа** — как записываем алгоритм так, чтобы машина смогла его исполнить.

Без представления нет ни программы, ни датасета. Алгоритм сортирует, ищет, обучает модель — но всегда на **дискретных** объектах: конечное число состояний, конечный объём. Data Science работает с большим объёмом кодированных признаков: бит и байт задают и размер выборки, и точность, а «качество данных» — это те же свойства информации, о которых ниже.

Эта лекция закрывает представление: что такое информация, как её мерить, как записывать целые числа. На семинаре появится язык программирования.

---

# 1. Информация и её измерение

## 1.1. Понятие информации

Есть вопрос, на который допустимо несколько ответов. Пока ответ неизвестен, есть **неопределённость**. Когда ответ получен, неопределённость уменьшилась.

**Информация** — снятие (уменьшение) неопределённости: было несколько допустимых исходов, стало меньше.

**Данные** — сведения, уже зафиксированные в конкретной форме: цифры, текст, таблица, набор битов. У данных есть объём и формат. Того, что они *кому-то* уменьшили неопределённость, данные сами по себе не гарантируют.

**Сообщение** — порция данных, переданная от источника к приёмнику.

**Пример (данные без информации).** Вопрос: «орёл или решка?» Честная монета, вы ещё не смотрели — два исхода, неопределённость есть. Вам прислали длинное письмо: «результат броска — орёл», но вы этот бросок уже видели. Данных много (письмо занимает байты), **информации для вас — ноль**: неопределённость уже была снята. Тот же текст для человека, который бросок не видел, информацию несёт. Информация зависит от получателя; данные — от того, как сообщение записано.

## Виды информации

Человек воспринимает мир разными каналами:

- **визуально** (текст, график, изображение);
- **на слух** (речь, звуковой сигнал);
- **тактильно** (брайль, жест).

По **форме представления** удобно различать текст, число, образ — и таблицу признаков (строка объекта, столбцы свойств). Датасет — большой набор таких таблиц и кодов.

Машине каналы восприятия безразличны. Звук, картинка и «да/нет» в памяти всё равно окажутся **дискретным кодом**. Разница видов важна человеку; для компьютера важен алфавит символов (или битов) и правила кодирования.

## Свойства информации

Свойства ниже — не мораль «информация должна быть хорошей», а критерии, которые потом всплывают в алгоритмах и в данных. Если на входе мусор, алгоритм и модель честно воспроизведут мусор.

**Пример-сюжет.** Таблица анализов пациентов: столбцы «диагноз», «температура», «дата анализа», «любимый цвет».

| Свойство             | Смысл                                 | В этом сюжете                                                                                     |
|----------------------|---------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>Достоверность</b> | Соответствует факту, без искажения    | В диагнозе опечатка или перепутан код болезни — метка в датасете неверная                         |
| <b>Полнота</b>       | Хватает сведений, чтобы решить задачу | Нет даты анализа — нельзя понять, к какому состоянию относится строка                             |
| <b>Актуальность</b>  | Сведения не устарели для этой задачи  | Дата есть, но анализу три года — для текущего решения бесполезно                                  |
| <b>Полезность</b>    | Сведения влияют на задачу             | «Любимый цвет» для постановки диагноза, скорее всего, не нужен: данные есть, на решение не влияют |
| <b>Понятность</b>    | Однозначны формат, единицы, кодировка | Температура 36,6 без шкалы: °C или °F? Без договорённости строка двусмысленна                     |

Для курса дальше особенно важны **полнота**, **актуальность** и **понятность** (однозначность представления). Без них нельзя ни корректно поставить алгоритм, ни собрать признаковое описание объекта.

## 1.2. Измерение информации

Количество информации можно мерить по-разному. Разберём два подхода.

### Содержательный подход

Смотрим, **сколько неопределённости сняли** для получателя.

Пусть есть  $N$  **равновозможных** исходов, и сообщение указывает ровно один из них. Тогда количество информации (формула Хартли)

$$I = \log_2 N \quad (\text{бит}).$$

Если исход получателю уже известен, содержательно  $I = 0$ , даже если текст сообщения длинный.

Если исходы **не** равновозможны, «сюрприз» другой: редкое событие снимает больше неопределённости, чем ожидаемое. Количественно это описывает формула Шеннона; на этой лекции её нет. Дальше в курсе работаем с равновозможным случаем и с «сырым» объёмом в битах.

### Пример 1. Монета и кубик.

- Честная монета:  $N = 2$ ,  $I = \log_2 2 = 1$  бит. Вопрос «да/нет» с двумя равновозможными ответами — тоже 1 бит.
- Честный кубик:  $N = 6$ ,  $I = \log_2 6 \approx 2,58$  бит. Столько информации в *содержательном* смысле несёт сообщение о грани. Чтобы **хранить** один из шести исходов в двоичных ячейках, ячеек должно быть целое число, не меньше  $\lceil \log_2 6 \rceil = 3$ . Три бита дают уже  $2^3 = 8$  кодов — с запасом.

Дробное  $\log_2 N$  и целое число ячеек  $\lceil \log_2 N \rceil$  — разные величины. Первая — количество информации, вторая — сколько бит нужно на код.

### Пример 2. Выбор из группы.

- Один студент из 32:  $N = 32 = 2^5$ ,  $I = 5$  бит, и на хранение тоже 5 бит.
- Один студент из 30:  $I = \log_2 30 \approx 4,91$  бит, на хранение  $\lceil \log_2 30 \rceil = 5$  бит ( $2^4 = 16 < 30 \leq 32 = 2^5$ ).

Так же оценивают дискретный признак в данных: категория из  $N$  значений содержательно несёт  $\log_2 N$  бит (если значения равновозможны), а в памяти под неё всё равно отводят целое число бит.

## Алфавитный подход

Смотрим не смысл, а **как сообщение записано**.

**Алфавит** — конечный набор символов, которыми пишут сообщения.

**Мощность алфавита**  $k$  — число символов в нём. **Длина** сообщения  $n$  — число позиций.

Число различных сообщений длины  $n$ :

$$N = k^n.$$

Если все такие сообщения (и все символы) считать равновозможными, объём по алфавитному подходу

$$I = \log_2 N = \log_2(k^n) = n \log_2 k \quad (\text{бит}).$$

На один символ:  $I_1 = \log_2 k$ . На сообщение длины  $n$ :  $I = n \cdot I_1$ .

Техника пользуется именно этим: «файл весит столько-то», независимо от того, осмыслен текст или нет.

### Пример 3. Слово над маленьким алфавитом.

Алфавит из 4 символов, слово длины 5:

$$N = 4^5 = 1024, \quad I = 5 \log_2 4 = 5 \cdot 2 = 10 \text{ бит.}$$

Бессмысленный набор из тех же 5 символов алфавитно занимает **те же** 10 бит. Содержательно он может не нести почти ничего — или нести, если получатель ждал именно этот код. В этом и разница подходов: алфавитный не спрашивает, удивились ли вы.

### Пример 4. Байт как алфавит из 256 знаков.

Алфавит из 256 символов:  $I_1 = \log_2 256 = 8$  бит на символ. Это мотивация **байта**: один байт кодирует любой из 256 знаков расширенного однобайтового алфавита. 8 бит и 256 значений — одно и то же.

### Пример 5. Число как код.

Целые от 0 до 1000 включительно — это  $N = 1001$  различных значений.

$$\log_2 1001 \approx 9,97 \text{ бит}, \quad \lceil \log_2 1001 \rceil = 10 \text{ бит на хранение}$$

$$(2^9 = 512 < 1001 \leq 1024 = 2^{10}).$$

Категориальный признак «номер от 0 до 1000» в таблице данных — тот же расчёт.

## Бит, байт, приставки

**Бит** (binary digit) — и минимальная единица количества информации (два равновозможных исхода), и ячейка с двумя состояниями: 0 или 1. Компьютер собирает все коды из битов, потому что физически удобны два устойчивых состояния.

**Байт** — группа из 8 бит. Число различных значений одного байта:

$$2^8 = 256$$

(от  $00000000_2$  до  $11111111_2$ , в десятичной — от 0 до 255). Не 255 значений, а **256**: нулевой код — тоже законное значение.

Крупные объёмы записывают с приставками. Есть два ряда, и их часто путают.

- Десятичные (СИ): кило- =  $10^3$ , мега- =  $10^6$ , гига- =  $10^9$ , ... экса- =  $10^{18}$ . Производители дисков обычно считают так: «1 Гбайт  $\approx 10^9$  байт».
- Двоичные (как степени 2, удобно памяти и ОС): 1 КиБ =  $2^{10} = 1024$  байт, 1 МиБ =  $2^{20}$ , 1 ГиБ =  $2^{30}$ , ... 1 ЭиБ =  $2^{60}$ .

В разговорной практике «килобайт» часто молча считают за 1024 байт. Насколько это расходится — видно на гигабайтах. Полная таблица — в справочнике в конце конспекта.

### Пример 6. Файл «3 Мбайт».

- Если мегабайт как  $2^{20}$ :  $3 \times 1\,048\,576 = 3\,145\,728$  байт.
- Если как  $10^6$ : 3 000 000 байт.

Разница около 5 %. На одном файле это мелочь, на датасете в сотни гигабайт — уже место на диске и время чтения.

---

## 2. Системы счисления

**Система счисления** — правило записи чисел цифрами.

**Позиционная** система с основанием  $b$  ( $b \geq 2$ , целое): значение цифры зависит от места. Запись  $d_m d_{m-1} \dots d_1 d_0$  означает

$$\sum_{i=0}^m d_i \cdot b^i, \quad 0 \leq d_i < b.$$

Число — не «нарисованные цифры», а значение этого многочлена. Основание  $b$  в индексе:  $45_{10}$ ,  $101101_2$ ,  $2A_{16}$ .

**Непозиционная** (пример — римская): символ имеет фиксированный вклад, веса  $b^i$  нет.  $X$  всегда десять, не «десять или сто в зависимости от колонки».  $XIV$  и  $14$  обозначают одно и то же, но римскую запись неудобно складывать столбиком и почти невозможно отдать процессору как набор однотипных разрядов. Поэтому машины работают с позиционными системами.

## Основание 10

Привычная запись. Цифры  $0, \dots, 9$ , вес разряда — степень десяти:

$$205_{10} = 2 \cdot 10^2 + 0 \cdot 10^1 + 5 \cdot 10^0.$$

Ноль в середине важен: он держит разряд. Это тот же принцип, что потом в двоичной.

## Основание 2

Цифры только 0 и 1 — ровно состояния бита. Поэтому двоичная система естественна для компьютера.

$$13_{10} = 1101_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 1.$$

Весы разрядов справа налево:  $1, 2, 4, 8, 16, \dots$  — степени двойки. Таблица степеней есть в справочнике.

## Основание 16 (шестнадцатеричная, hex)

Цифры  $0, \dots, 9$  и  $A, B, C, D, E, F$  для значений  $10, \dots, 15$ .  $F$  — это **одна** цифра со значением 15, не «число 15 из двух десятичных цифр».

Человеку двоичная строка из десятков нулей и единиц плохо читается. Шестнадцатеричная короче и привычна в документации, дампах памяти, цветовых кодах.

## Основание 8 (восьмеричная)

Цифры  $0, \dots, 7$ . Ещё одно основание, исторически связанное с двоичной ( $8 = 2^3$ ). В современном программировании встречается реже hex. Перевод — снова только через 10.

## Два алгоритма перевода

Других схем в этом курсе не требуется. Чтобы перевести, например, из двоичной в шестнадцатеричную: сначала в десятичную, затем из десятичной.

### Из основания $b$ в десятичную: сумма весов

Подставить цифры в  $\sum d_i b^i$ . Это вычисление многочлена; математически вы это уже умеете.

**Пример.**  $11011101_2 \rightarrow 10$ .

Разряды справа налево, веса  $2^0, \dots, 2^7$ :

$$\begin{aligned} 11011101_2 &= 1 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 \\ &\quad + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 128 + 64 + 0 + 16 + 8 + 4 + 0 + 1 = 221_{10}. \end{aligned}$$

Дальше, если нужна hex:  $221_{10} \rightarrow 16$  делением (следующий алгоритм).

$221 : 16 = 13$  (остаток  $13 = D$ ),  $13 : 16 = 0$  (остаток  $13 = D$ ). Остатки снизу вверх:  $DD_{16}$ .

Проверка:  $13 \cdot 16 + 13 = 208 + 13 = 221$ .

### Из десятичной в основание $b$ : деление, остатки снизу вверх

Пока число не стало нулём: делим нацело на  $b$ , записываем **остаток**. Новая «целая часть» — частное. Готовую запись читаем **от последнего остатка к первому** (снизу вверх): последний остаток — старшая цифра.

**Пример.**  $45_{10} \rightarrow 2, 8, 16$ .

В двоичную ( $b = 2$ ):

| Деление | Частное | Остаток |
|---------|---------|---------|
| 45 : 2  | 22      | 1       |
| 22 : 2  | 11      | 0       |
| 11 : 2  | 5       | 1       |
| 5 : 2   | 2       | 1       |
| 2 : 2   | 1       | 0       |
| 1 : 2   | 0       | 1       |

Снизу вверх:  $101101_2$ .

Проверка суммой весов:  $32 + 8 + 4 + 1 = 45$ .

В восьмеричную ( $b = 8$ ):  $45 : 8 = 5$  (остаток 5),  $5 : 8 = 0$  (остаток 5).

Получаем  $55_8$ . Проверка:  $5 \cdot 8 + 5 = 45$ .

В шестнадцатеричную ( $b = 16$ ):  $45 : 16 = 2$  (остаток  $13 = D$ ),  $2 : 16 = 0$  (остаток 2). Получаем  $2D_{16}$ . Проверка:  $2 \cdot 16 + 13 = 45$ .

**Пример.**  $2A_{16} \rightarrow 10 \rightarrow 2$  (не «каждая хех-цифра сразу в биты»).

$$2A_{16} = 2 \cdot 16^1 + 10 \cdot 16^0 = 32 + 10 = 42_{10}.$$

$42_{10} \rightarrow 2$ :

| Деление | Частное | Остаток |
|---------|---------|---------|
| 42 : 2  | 21      | 0       |
| 21 : 2  | 10      | 1       |
| 10 : 2  | 5       | 0       |
| 5 : 2   | 2       | 1       |
| 2 : 2   | 1       | 0       |
| 1 : 2   | 0       | 1       |

Снизу вверх:  $101010_2$ . Проверка:  $32 + 8 + 2 = 42$ .

**Пример.**  $157_8 \rightarrow 10 \rightarrow 2$ .

$$157_8 = 1 \cdot 8^2 + 5 \cdot 8^1 + 7 \cdot 8^0 = 64 + 40 + 7 = 111_{10}.$$

$111_{10} \rightarrow 2$ : остатки при делении на 2 (по порядку делений)  
1, 1, 1, 1, 0, 1, 1 — читать снизу:  $1101111_2$ .

Проверка:  $64 + 32 + 8 + 4 + 2 + 1 = 111$ .

---

### 3. Зачем это программированию, алгоритмам и данным

**Программа.** Машина оперирует целыми числами и байтами. На семинаре те же позиционные записи можно будет набрать как  $0b1011$  (двоичная) и  $0x2A$  (шестнадцатеричная). Это не новый вид чисел, а другой алфавит цифр для того же значения.

**Алгоритм.** Он работает с конечным набором состояний.  $n$  бит дают не больше  $2^n$  различных значений. Отсюда позже: ограниченный диапазон целого типа, переполнение, необходимость явно выбрать представление. «Математическое целое» и «целое в машине» совпадают только внутри диапазона кода.

**Данные (Data Science).** Признак «категория из 5 значений» содержательно несёт не больше  $\log_2 5 \approx 2,32$  бит, если значения равновозможны. В памяти под него часто отводят целый байт (8 бит) или ещё шире — слово процессора. Полезной информации меньше, чем занятого объёма: алфавитный размер кода и содержательный объём признака не обязаны совпадать.

Объём выборки считают в байтах и гигабайтах — те же приставки, что в §1. Картинка без сжатия: каждый пиксель — код из нескольких байт ( $k^n$  и  $2^8$  снова). Качество датасета — достоверность меток, полнота колонок, актуальность, однозначный формат: те же свойства информации.

---

## Справочник

Сюда заглядывают, когда считают. Это не сюжет лекции.

## Степени двойки

| $n$ | $2^n$ | $n$ | $2^n$  |
|-----|-------|-----|--------|
| 0   | 1     | 9   | 512    |
| 1   | 2     | 10  | 1 024  |
| 2   | 4     | 11  | 2 048  |
| 3   | 8     | 12  | 4 096  |
| 4   | 16    | 13  | 8 192  |
| 5   | 32    | 14  | 16 384 |
| 6   | 64    | 15  | 32 768 |
| 7   | 128   | 16  | 65 536 |
| 8   | 256   |     |        |

Отдельно:  $2^{10} = 1\,024$ ,  $2^{20} = 1\,048\,576$ ,  $2^{30} = 1\,073\,741\,824$ .

## Цифры 0–15

Справочник значений цифр (не метод перевода группировкой бит).

| dec | bin (4 бита) | oct | hex |
|-----|--------------|-----|-----|
| 0   | 0000         | 0   | 0   |
| 1   | 0001         | 1   | 1   |
| 2   | 0010         | 2   | 2   |
| 3   | 0011         | 3   | 3   |
| 4   | 0100         | 4   | 4   |
| 5   | 0101         | 5   | 5   |
| 6   | 0110         | 6   | 6   |
| 7   | 0111         | 7   | 7   |
| 8   | 1000         | 10  | 8   |
| 9   | 1001         | 11  | 9   |
| 10  | 1010         | 12  | A   |
| 11  | 1011         | 13  | B   |
| 12  | 1100         | 14  | C   |
| 13  | 1101         | 15  | D   |
| 14  | 1110         | 16  | E   |
| 15  | 1111         | 17  | F   |

## Единицы информации

| Единица | СИ (обычно «десятичные») | Двоичные приставки    |
|---------|--------------------------|-----------------------|
| бит     | минимальная единица      | то же                 |
| байт    | 8 бит                    | то же                 |
| кило-   | 1 Кбайт = $10^3$ байт    | 1 КиБ = $2^{10}$ байт |
| мега-   | 1 Мбайт = $10^6$ байт    | 1 МиБ = $2^{20}$ байт |
| гига-   | 1 Гбайт = $10^9$ байт    | 1 ГиБ = $2^{30}$ байт |
| тера-   | 1 Тбайт = $10^{12}$ байт | 1 ТиБ = $2^{40}$ байт |
| пета-   | 1 Пбайт = $10^{15}$ байт | 1 ПиБ = $2^{50}$ байт |
| экса-   | 1 Эбайт = $10^{18}$ байт | 1 ЭиБ = $2^{60}$ байт |

Накопители часто маркируют по столбцу СИ; объём в свойствах файла в ОС — часто по степеням 2. Отсюда «диск 500 Гбайт, а в системе меньше».

## Шпаргалка перевода

$b \rightarrow 10$ . Расписать  $\sum d_i b^i$  и сложить. Проверка: оценка порядка ( $b^m$  чуть меньше результата).

$10 \rightarrow b$ . Делить нацело на  $b$ , писать остатки, читать **снизу вверх**. Проверка: полученную запись снова перевести в десятичную суммой весов.

Перевод  $2 \leftrightarrow 8$ ,  $2 \leftrightarrow 16$ ,  $8 \leftrightarrow 16$ : сначала в десятичную, затем из неё. Других схем в этом курсе на зачёте не требуется.

---

## Контрольные вопросы

Короткие вопросы на понимание. Эталоны — в следующем разделе; сначала ответьте сами.

1. Чем информация отличается от данных? Приведите пример данных без информации для конкретного получателя.
2. Чем алфавитный подход к количеству информации отличается от содержательного?
3. Почему именно бит удобен как единица и для теории, и для компьютера?
4. Как связаны  $N = k^n$ ,  $I = \log_2 N$  и формула  $I = n \log_2 k$ ?
5. Что такое позиционная система счисления? Чем она удобнее непозиционной для вычислений?
6. Почему в компьютере естественно основание 2, а человеку за компьютером часто удобнее основание 16?
7. Сколько различных значений можно закодировать одним байтом? Почему 256, а не 255?
8. Как перевести целое число из десятичной системы в систему с основанием  $b$ ? Как перевести из основания  $b$  в десятичную?

9. Чем «кило» в  $10^3$  отличается от «кило» в  $2^{10}$ , и когда эта разница всплывает на практике?
10. Почему объём данных в памяти может быть больше «информационного содержания» признака (пример с категорией из нескольких значений)? Зачем алгоритму и модели фиксировать представление (диапазон, дискретность)?

## Ответы для самопроверки

1. Данные — зафиксированная запись (символы, биты, таблица).  
Информация — уменьшение неопределённости у получателя.  
Пример: подробное письмо о результате броска монеты человеку, который этот бросок уже видел: байты есть, неопределённость не уменьшилась.
2. Содержательный смотрит, сколько исходов сняли (для равновозможных  $I = \log_2 N$ ; если уже знали — 0). Алфавитный смотрит только алфавит и длину ( $I = n \log_2 k$ ), смысл и ожидания не важны. Файл «весит» по алфавитному подходу.
3. В теории 1 бит — ответ на вопрос с двумя равновозможными исходами, минимальная порция. В машине удобны два устойчивых состояния ячейки; все коды собираются из таких ячеек. Одна единица служит и мере, и носителю.
4. Сообщений длины  $n$  над алфавитом мощности  $k$  ровно  $N = k^n$ . Тогда  $\log_2 N = \log_2(k^n) = n \log_2 k$ . Это один и тот же объём, записанный через число сообщений или через длину и алфавит.
5. Позиционная: вклад цифры = значение  $\times$  вес  $b^i$  в зависимости от места. Непозиционная (римская) фиксирует вклад символа без таких весов. Позиционную удобно складывать поразрядно и отдавать машине как однотипные разряды; римскую — нет.
6. Основание 2 совпадает с битом (два состояния). Основание 16 даёт более короткую запись того же целого, её проще читать в документации; значение числа при этом то же. Перевод между ними на этой лекции — через десятичную систему.
7.  $2^8 = 256$  значений (коды 0...255). Нулевой код — тоже значение; «от 1 до 255» было бы 255 вариантов и не покрыло бы все комбинации 8 бит.

8. Из 10 в основание  $b$ : делить на  $b$ , остатки читать снизу вверх. Из  $b$  в 10: сумма  $\sum d_i b^i$ . Из 2 в 16 (и любой пары 2, 8, 16): сначала в 10, затем из 10.
9.  $10^3 = 1000$ ,  $2^{10} = 1024$ . «Килобайт» в рекламе диска часто  $10^3$  байт (и дальше мега-/гига- как  $10^{3n}$ ); в памяти и во многих ОС — степени 2. На гигабайтах расхождение уже заметно: «на коробке больше, в системе меньше».
10. Категория из 5 значений содержательно  $\leq \log_2 5$  бит, а хранят её байтом или словом — алфавит кода шире, чем нужно по Хартли. Алгоритму и модели нужно фиксированное представление: конечный диапазон, дискретная сетка. Иначе нельзя ни отвести память, ни гарантировать, что «вещественное математическое» значение вообще существует в машине как код.